

Cerinta 1. Exploratory Data Analysis

1. Analiza tipului de atribute si a plajei de valori a acestora

Statistici pentru attributele numerice continue:

Feature	Non null	mean	std	min	25%	50%	75%	max
experience_years	64000	9.97	6.05	0	5	10	15	20
skills_count	64000	11.6	10.74	1	5	10	15	69
certifications	64000	2.51	1.71	0	1	3	4	5
total_days_worked	64000	2393.51	1452.04	-1	1200	2400	3600	4800
aggregated_score	64000	0	1	-4.8	-0.67	0	0.68	4.22
salary	64000	145683.57	37384.76	31867	119209.75	143299	169582.25	333046

Tabel 1 – Statistici descriptive ale atributelor numerice continue din setul de antrenare

Din statistici, se observa ca **skills_count** are o distributie aproape uniforma in [1, 19] cu ~3200 de exemple pe fiecare valoare, urmata de un gol total in [20, 59] si o a doua zona cu 200 de exemple pe valoare la [60, 69]. Faptul ca avem un gol de oameni in zona de skill-uri [20, 59] poate indica ca datele nu au fost colectate corect. De asemenea, 200 de oameni per valoare declara skills_count intre [60, 69] poate sugera aparitia unor outliers. Acesti outliers dau impresia de distributie bimodala, si s-ar putea sa afecteze regresia liniara si clasificarea mai incolo daca nu sunt tratati.

In alta ordine de idei, intre **experience_years** si **total_days_worked** se observa o relatie matematica:

$\text{total_days_worked} = 240 * \text{experience_years}$, ceea ce sugereaza o redundanta.

Aggregated score este aproape z-score, 0 pentru mean, 1 pentru standard deviation, dar are corelatia Pearson aproape nula cu toate attributele, deci, ar trebui eliminat la antrenare.

Alte corelatii: am observat corelatia Pearson intre **experience_years** si **salary** este de 0.44 (cea mai mare corelatie numerica); Cramer's V intre **education_level** si **vacation** este de 0.32 si **company_size** cu **vacation** este de 0.24 – cei mai puternici predictorii categorici.

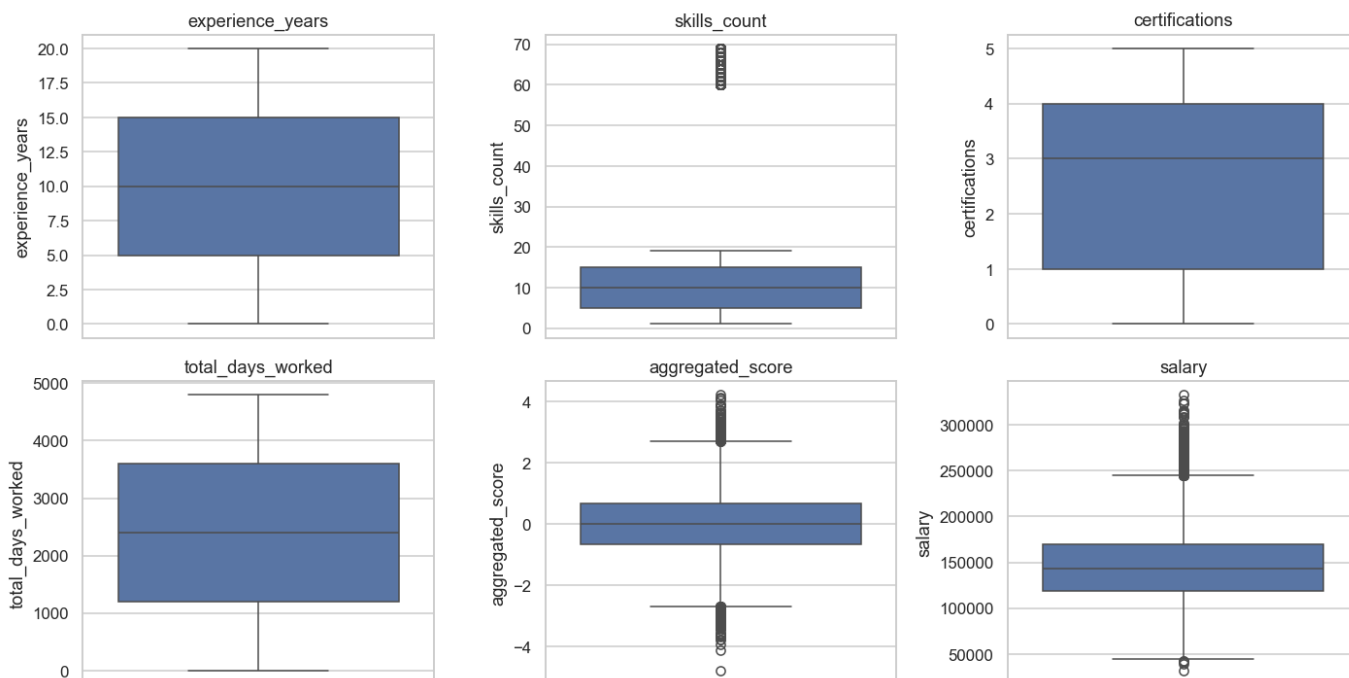


Fig. 1 – Grafice de tip Boxplot pentru plajele de valori ale atributelor numerice continue

column	non_null	missing	unique	top	top_freq
job_title	64000	0	12	AI Engineer	5410
industry	64000	0	10	Consulting	6547
company_size	64000	0	5	Large	12934
location	64000	0	10	Sweden	6570
remote_work	44847	19153	3	Hybrid	15054
vacation	64000	0	4	No Vacation	34686
education_level	64000	0	5	Bachelor	12863
skill_bracket	64000	0	3	low	21470

Tab. 2 – Atribute discrete/categoriale primele 6 si ordinale ultimele 2

Am extras din setul de date numarul exemplelor care nu au valori lipsa, care au valori lipsa, care au valori unice. De asemenea, am extras si valoarea cea mai frecventa din setul de date pentru attributele respective, si frecventa acesteia.

Se poate observa un **dezechilibru masiv** intre clase pentru variabila **vacation**, clasa No Vacation avand 54.19% din exemple, in timp ce celelalte clase au mult mai putin, Small 20.12%, Medium 13.32%, Large 12.35%. Asta va fi o problema pentru clasificare, deoarece modelul nu are suficiente date pentru fiecare clasa.

Se poate observa ca pentru atributul **remote_work**, o portiune semnificativa de exemple din setul de date are valori lipsa, respectiv ~30%.

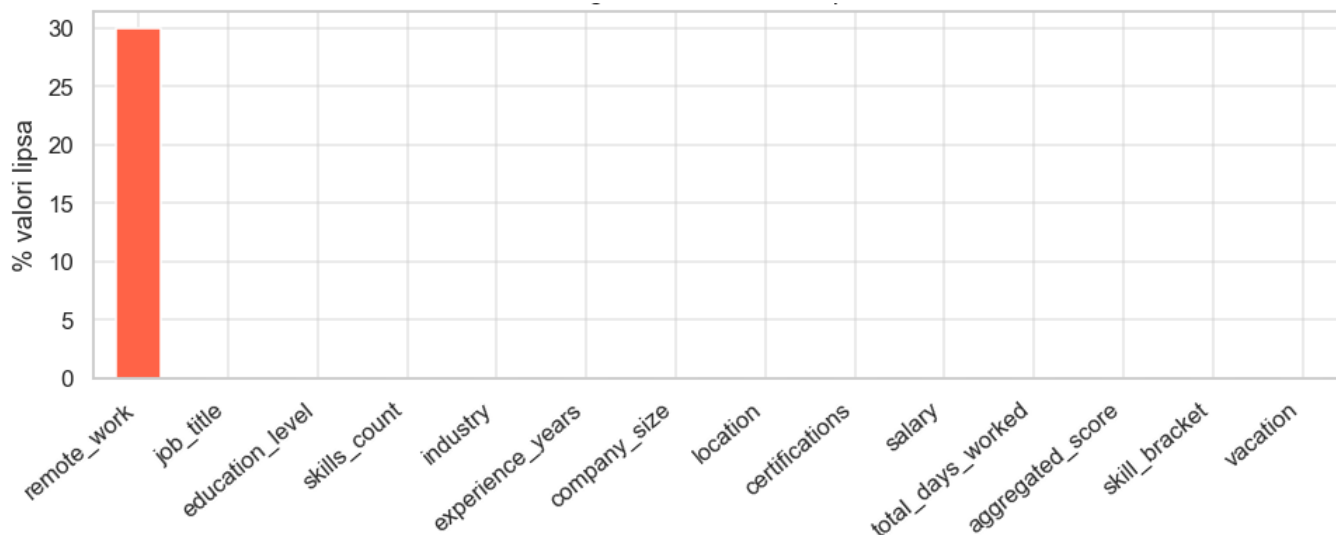
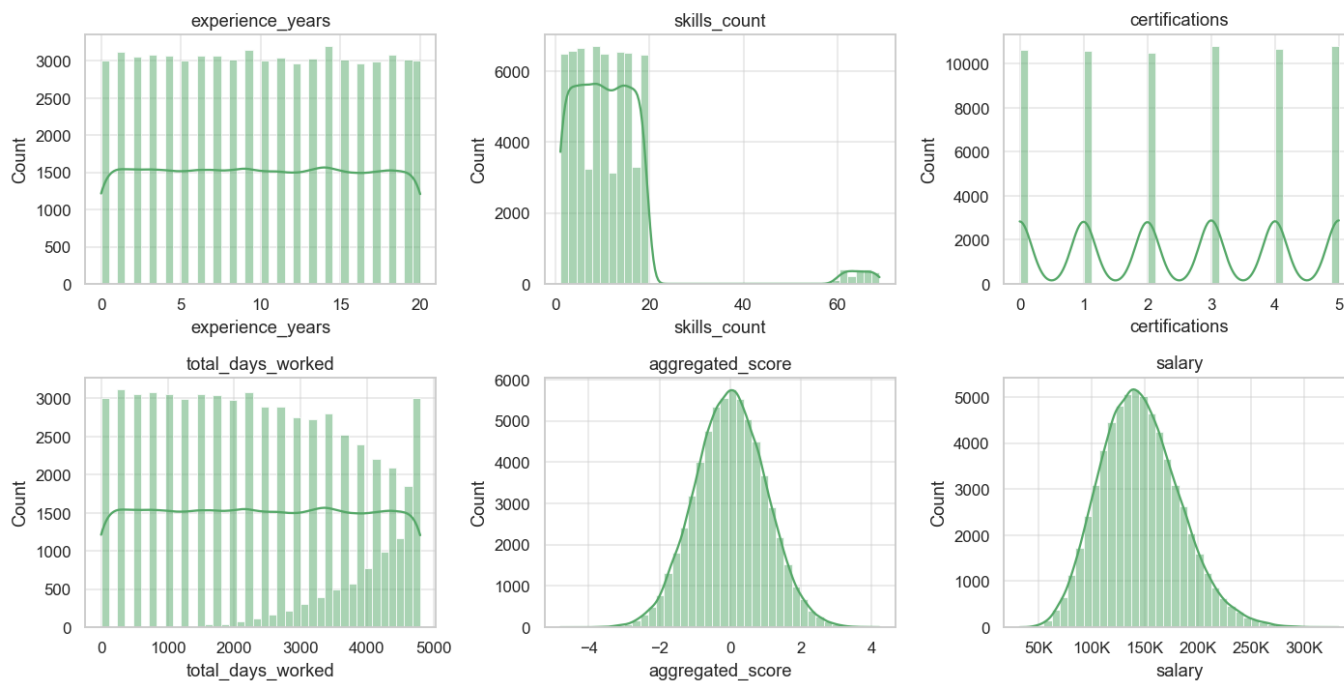
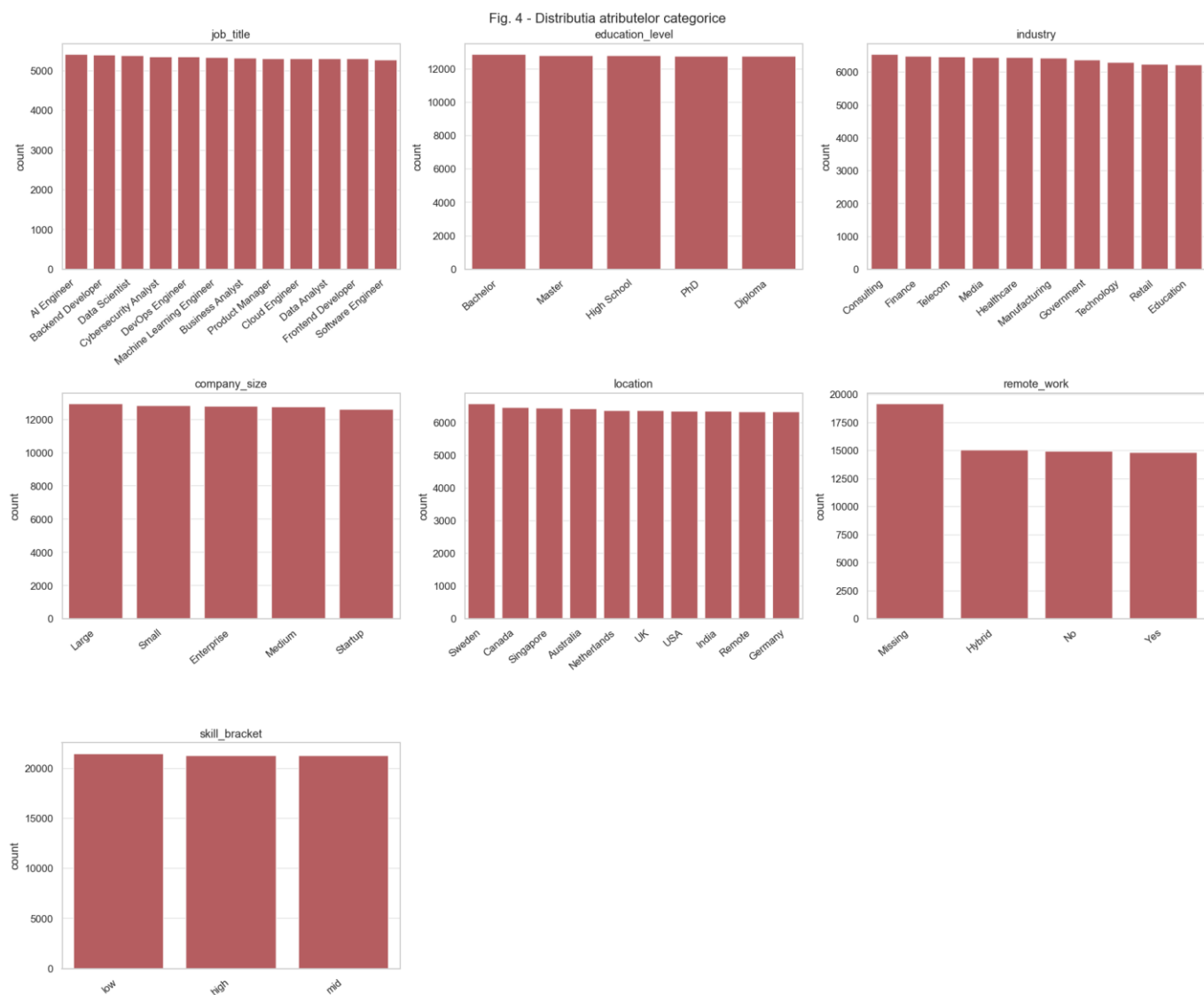


Fig. 2 – Procentul exemplelor cu valori lipsa din setul de date.

Fig. 3 - Distributia atributelor numerice



Am trecut valorile salariilor in mii in loc de unitati pentru a incapa mai bine.



In Fig. 4, se poate vedea ca pentru variabila **remote_work**, celelalte clase au aceeași distribuție a datelor, respectiv Hybrid / No / Yes.

In Fig. 5, se poate observa ca pentru variabila vacation, dezechilibrul este prezent atat in setul de antrenare, cat si in cel de validare.

Fig. 5 - Dezechilibrul de clase pentru 'vacation' pt antrenare

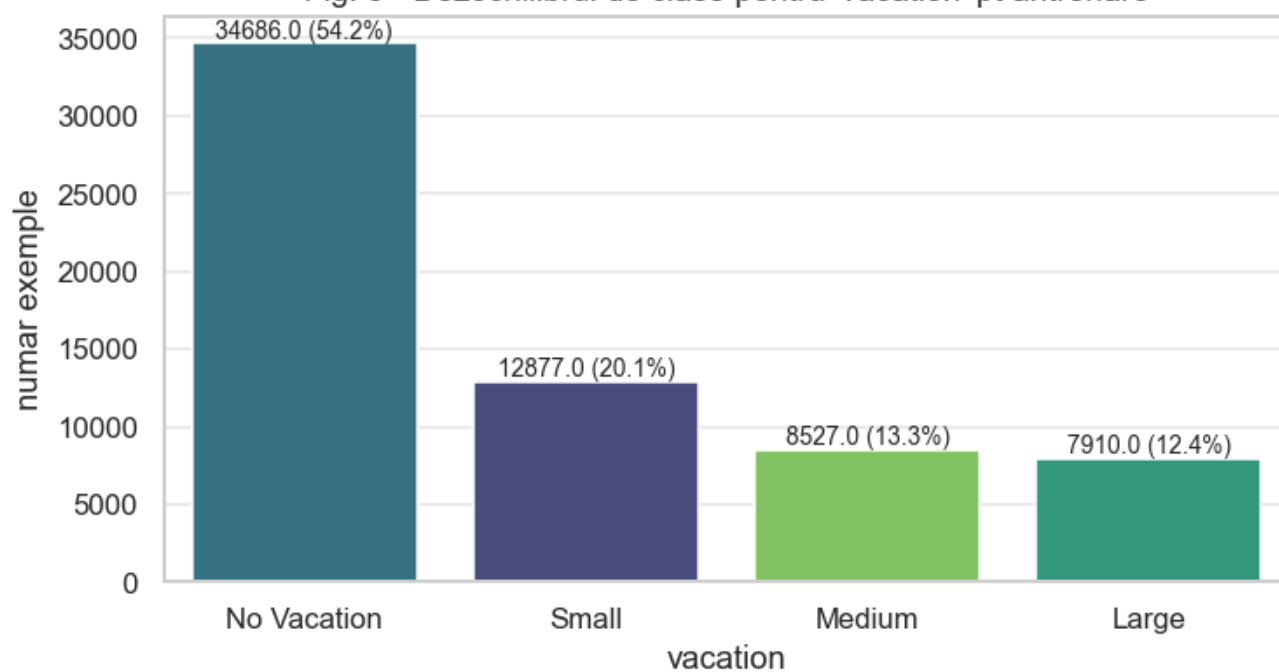


Fig. 5 - Dezechilibrul de clase pentru 'vacation' pt validare

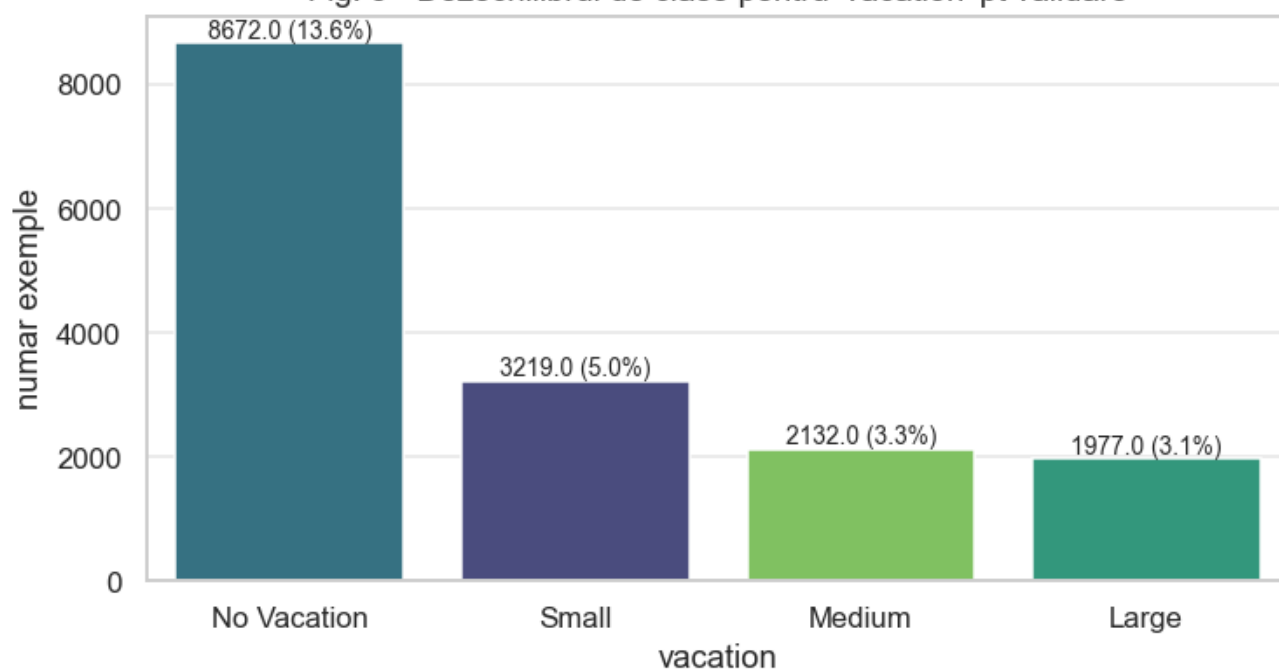
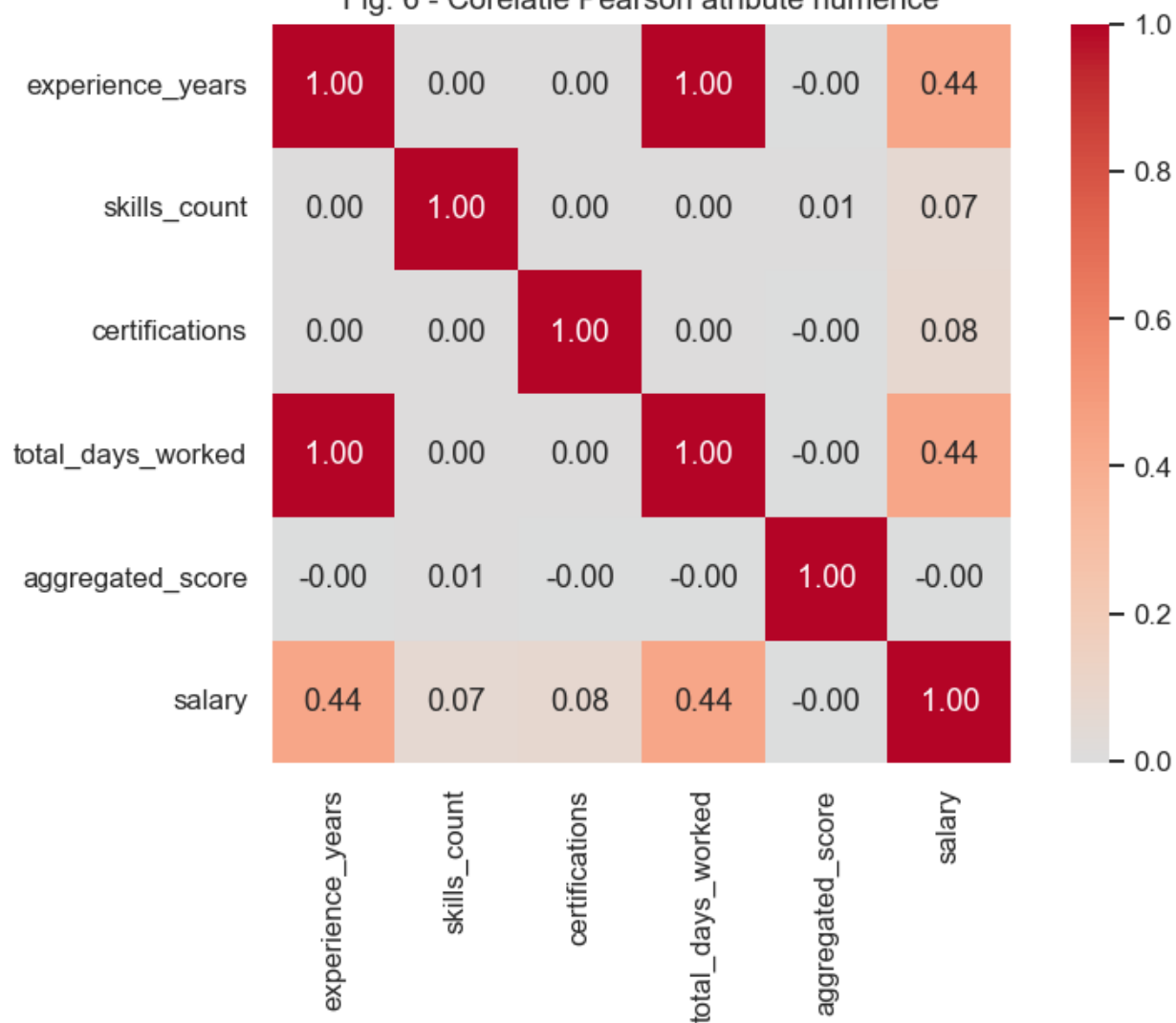
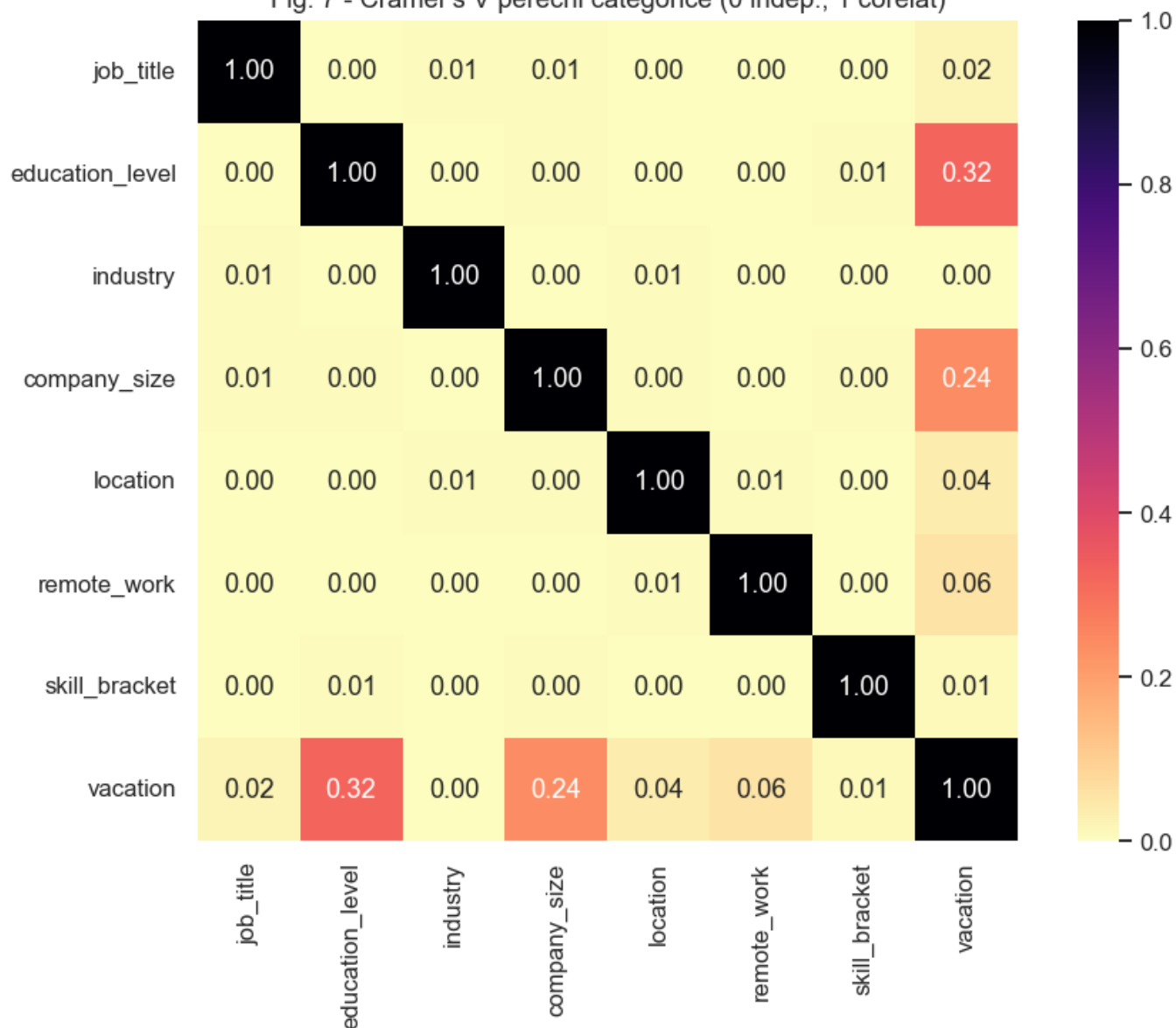


Fig. 6 - Corelatie Pearson attribute numerice



Din corelatia Pearson, se poate observa ca total_days_worked este perfect corelat cu experience_years, fapt sugerat si de observatia matematica facuta mai devreme.

Fig. 7 - Cramer's V perechi categorice (0 indep., 1 corelat)

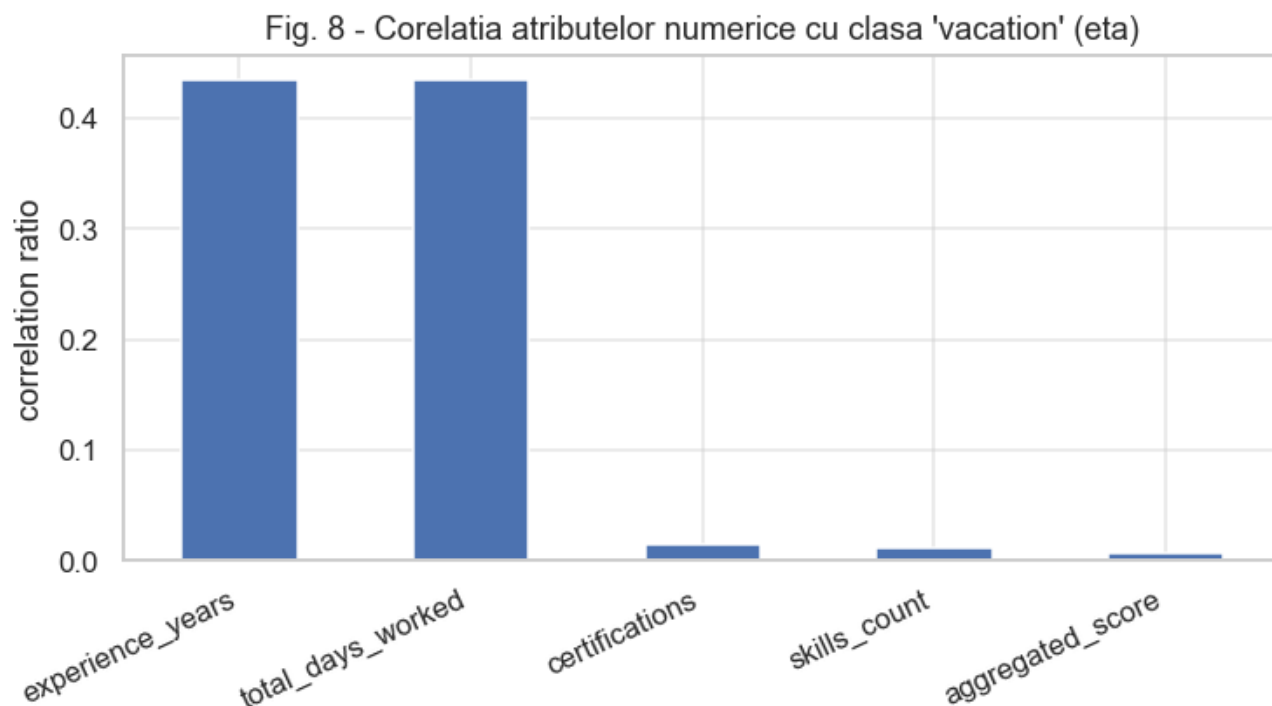


Corelatia Cramer's V este un coeficient statistic pentru a masura puterea corelatiei intre 2 variabile categorice. Se poate observa ca intre vacation si education_level exista o corelatie medie, la fel cum exista si intre company_size si vacation.

M-am gandit la o posibila **corelatie intre skill_bracket si skills_count**, si am testat aceasta ipoteza. Daca intervalele skill_bracket erau disjuncte, atunci puteam vorbi despre faptul ca skill_bracket este o discretizare a lui skills_count. Dar, toate intervalele sunt afectate de outlierii din skills_count:

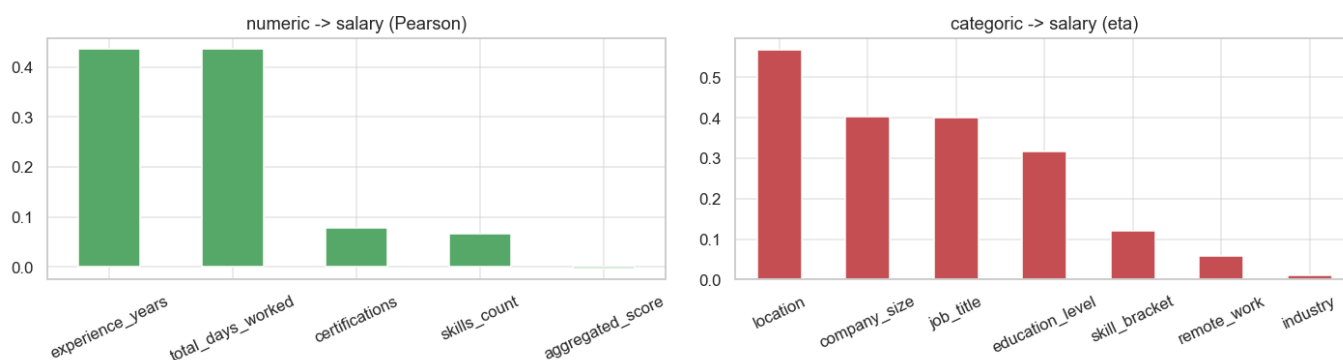
skill_bracket	min	max	count
low	1	69	21470

mid	7	69	21253
high	13	69	21277



Atributul **experience_years** este cel mai **puternic predictor numeric** pentru sarcina de clasificare, având un raport de corelație de aproximativ **0.43**.

Fig. 9 - Corelatia atributelor cu tinta de regresie 'salary'



Deoarece **education_level** si **company_size** influenteaza direct salariul intr-o masura semnificativa, acestea devin **predictori categorici esentiali pentru clasificarea Vacation**. De ex, un angajat cu PhD intr-o companie de tip Enterprise are o probabilitate mai mare sa apartina clasei Large pt tinta de clasificare Vacation, decat unul cu education_level High School dintr-un Startup.

Intre `experience_years` si `salariu` se poate observa o **corelatie puternica Pearson**. Cu cat cineva are mai multa experienta, cu atat salariul e mai mare. In fig. 14 se confirma acest lucru prin faptul ca norul de puncte urca (salariul creste) pe masura ce experienta creste. Totusi, dispersia asa de mare a punctelor sugereaza ca nu putem folosi doar experienta pentru a prezice salariul, respectiv pentru a minimiza MSE.

Se poate observa ca **location** este cel mai puternic atribut categorial pentru sarcina de **regresie**, cu un coeficient eta de peste 0.55.

Fig. 10 - Salariu in functie de attribute cheie

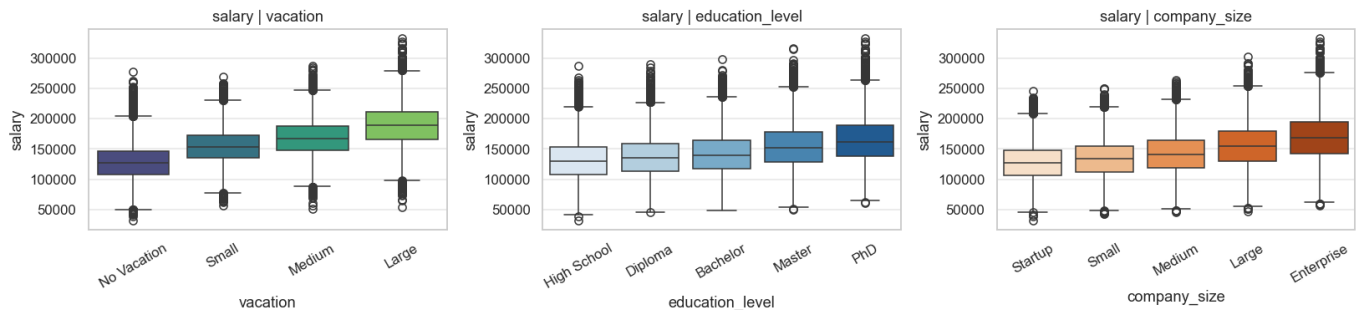
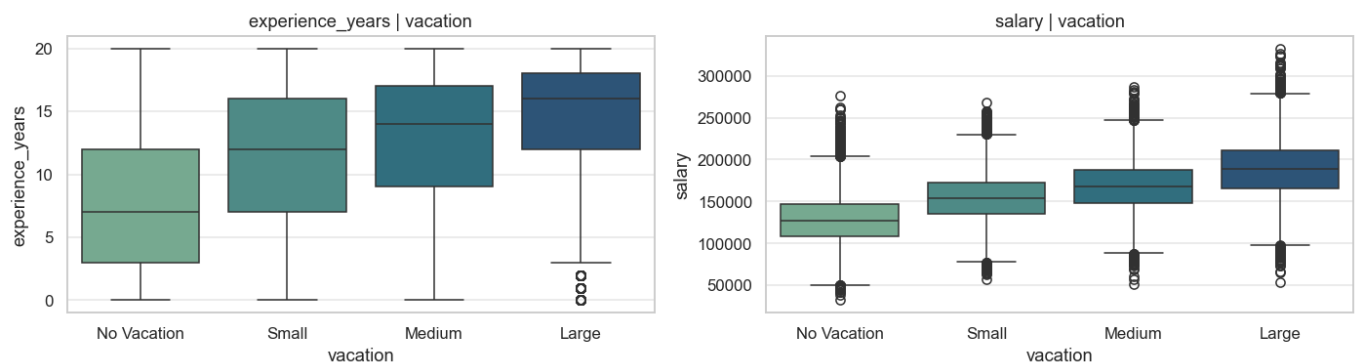
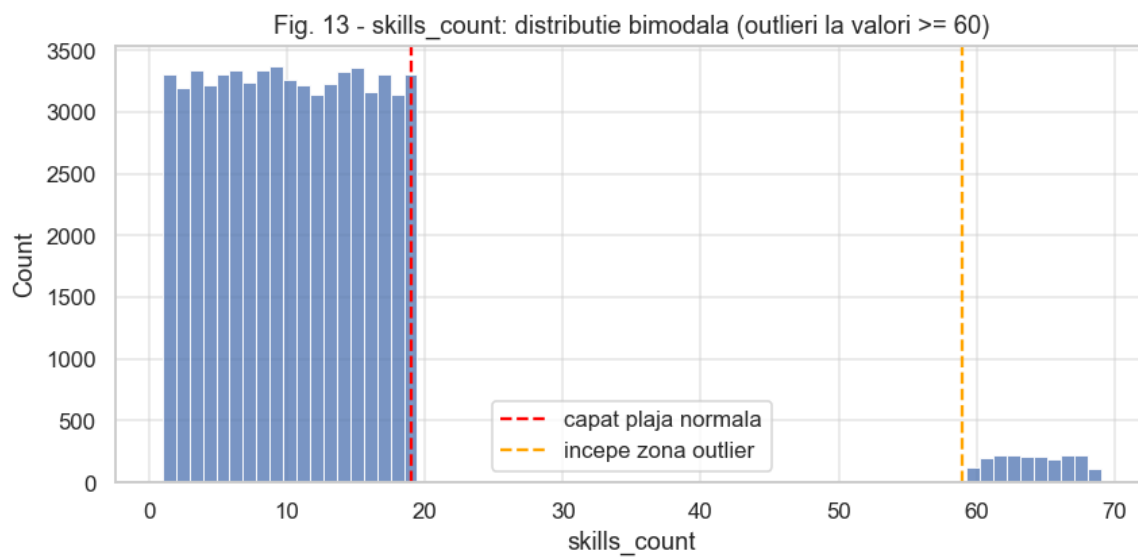
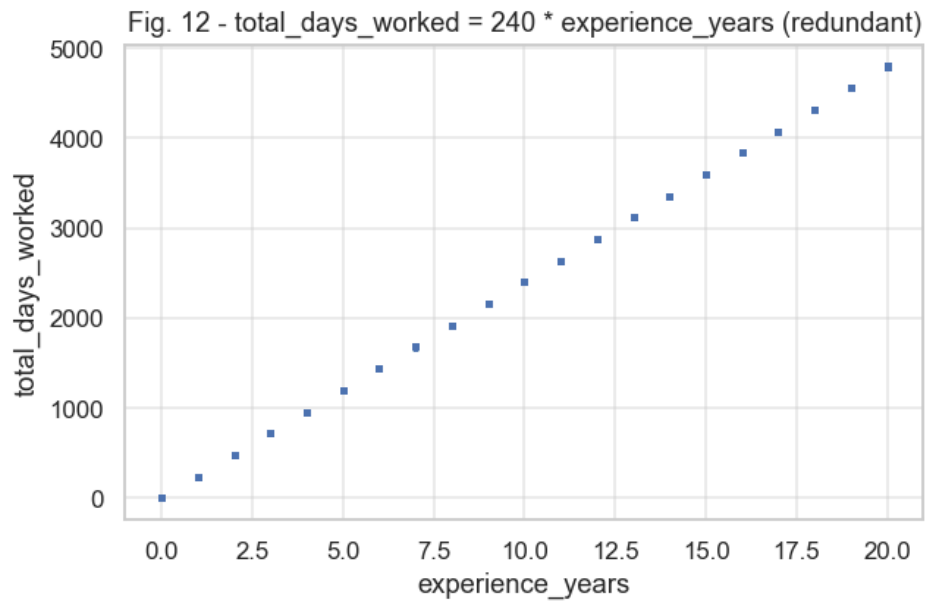


Fig. 11 - Attribute numerice vs clasa 'vacation'

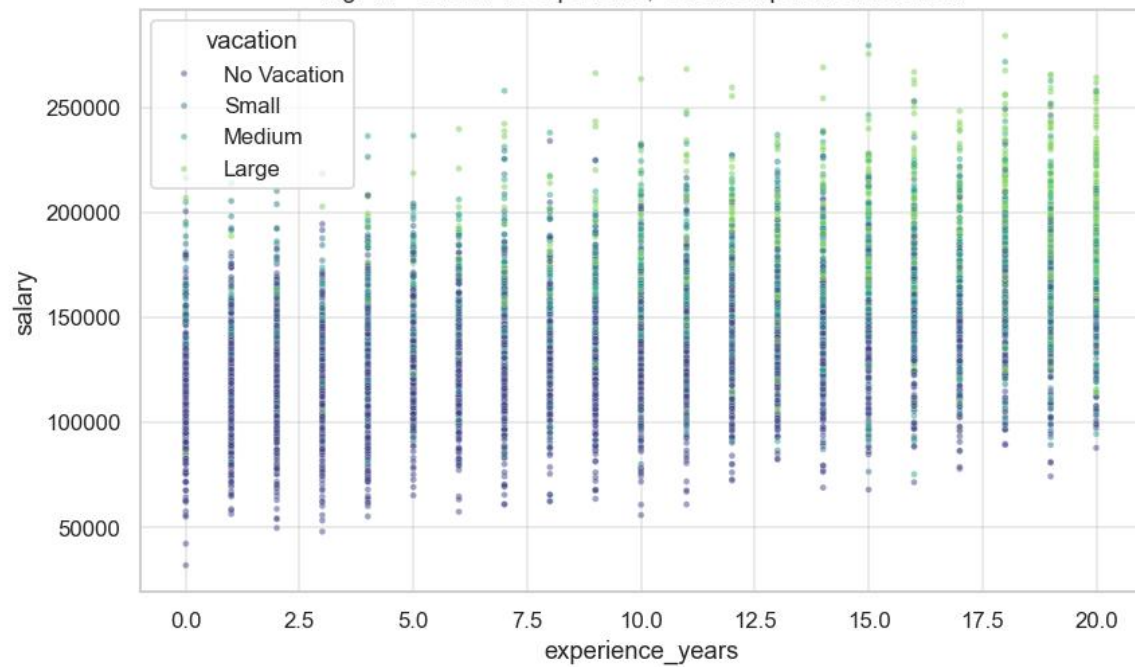


Se observa clar ca angajatii cu experienta ridicata (mediana peste 15 ani) tind sa fie clasificati in categoria “Large Vacation”, in timp ce cei cu experienta redusa domina clasa “No Vacation”. Aceasta relatie monotona sugereaza ca experienta este o caracteristica de baza pe care modelul o va folosi pentru primele split-uri. Salariul creste strict cu nivelul de vacation.



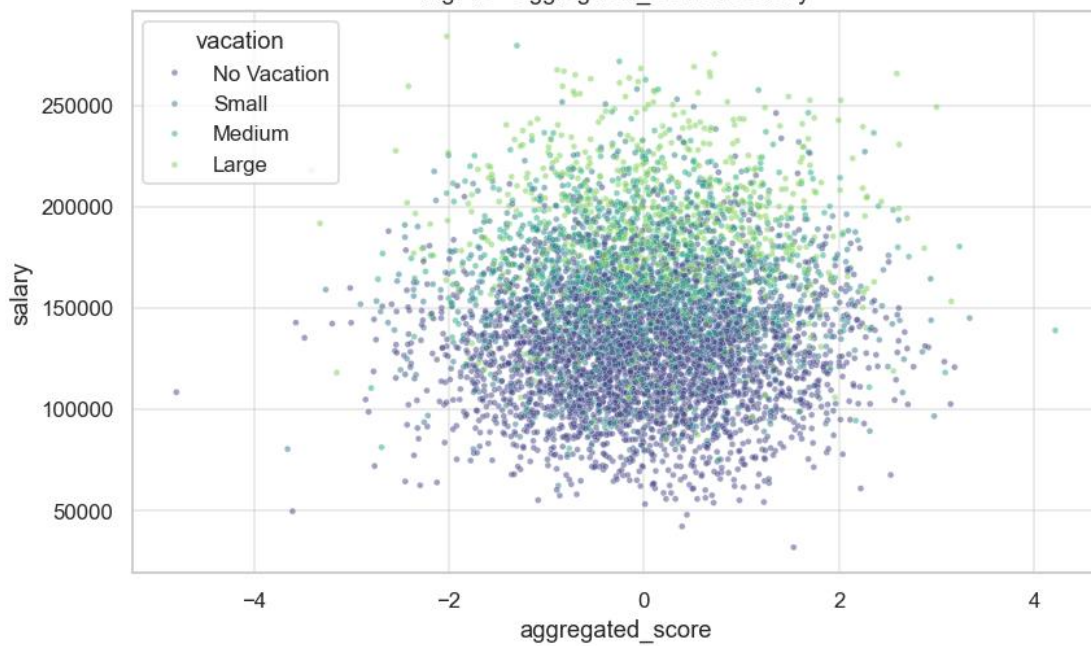
In fig. 13, se vede clar ca skills_count are outliers la valori mai mari de 60.

Fig. 14 - salariu vs experienta, colorat dupa clasa vacation



Claselor sunt separate destul de clar dupa salariu.

Fig. 15 - aggregated_score vs salary



Nu e nicio corelatie intre aggregated_score si salary. E doar zgomot.

Cerinta 2 – Preprocesarea datelor

Toate transformarile au fost invatate pe setul de antrenare si aplicate identic pe setul de validare / test.

1. Imputarea valorilor lipsa

a) Atribute numerice (SimpleImputer cu strategia median)

Pentru atributele numerice, am ales mediana in locul mediei deoarece atribute precum *skills_count* prezinta distributii asimetrice si outliers (Fig. 13). Si in general, daca pt atributul *salary* avem un outlier cu o valoare foarte mare, acesta ar duce media foarte sus, in timp ce mediana ar ramane la fel. Acelasi rationament se aplica si pentru *experience_years* si *certifications*.

b) Atribute categorice (SimpleImputer cu strategia mode – cea mai frecventa valoare)

Singura coloana cu valori lipsa este cea pentru **remote_work**, avand un procent de aproximativ 30% date lipsa. Am imputat folosind mode (cea mai frecventa valoare), care este "Hybrid". Aceasta alegere este mai conservatoare decat stergerea randurilor (ar insemna pierderea a 30% din date) si mai neutra decat o imputare aleatoare. Varianta alternativa de imputare ar fi fost adaugarea unei categorii distincte "Unknown" pentru a pastra informatia ca valoarea lipsa. In urma testarii, ambele variante au produs rezultate similare, deci am mers cu mode ca solutie mai simpla.

2. Tratarea valorilor extreme pentru un atribut intr-un esantion

Am utilizat metoda **Interquartile Range (IQR)** pentru a detecta valorile care depasesc limitele de $Q1 - 1.5 * IQR$ si $Q3 + 1.5 * IQR$. Valorile identificate ca outliers au fost plafonate prin inlocuirea lor cu mediana coloanei respective. Desi tratarea outlierilor este recomandată pentru algoritmi liniari, am descoperit ca pentru **Regresie**, capping-ul a dus la o crestere a erorii MSE. De asemenea, *total_days_worked* contine cateva valori de -1, pe care le-am corectat la 0. Desi tratarea outlierilor este recomandata pentru algoritmi liniari, experimentele de ablatie au aratat ca pentru **Regresie**, capping-ul a dus la o crestere a erorii MAE de la 6.305 la 7.058 (+12%), iar R^2 a scazut de la 0.9514 la 0.9246. Explicatia este ca outlieri precum salariile foarte mari sunt **reali** in dataset si nu reprezinta erori de masurare, iar eliminandu-i, modelul pierde informatia despre angajatii bine platiti. Din acest motiv, **nu am aplicat outlier capping in pipeline-ul final** pentru regresie.

3. Eliminarea atributelor redundante

In urma analizei corelatiei Pearson (fig. 6) si a raportului de corelatie eta (fig. 9), am eliminat 2 atribute pentru a simplifica modelarea si a evita overfitting-ul.

Am eliminat **total_days_worked**, intrucat este o corelatie directa cu *experience_years* ($total_days_worked = 240 * experience_years$), deci nu aduce informatie noua.

Am eliminat de asemenea si **aggregated_score**, intrucat este zgomot pur. In fig.15 se vede un nor de puncte fara nicio structura in raport cu salariul sau vacanta, fapt confirmat si de corelatia Pearson, unde acesta este aproape 0.

4. Encodarea atributelor categorice

Am utilizat un **ColumnTransformer** din scikit-learn care aplica transformari diferite in functie de tipul semantic al fiecarui atribut:

a) Atribute ordinale – OrdinalEncoder:

Atribut	Ordine Encodata
Education_level	HS = 0, Diploma=1, BS = 2, MS = 3, PhD = 4
Company_size	Startup = 0, Small = 1, Medium = 2, Large = 3, Enterprise = 4
Skill_bracket	Low = 0, Mid = 1, High = 2

b) Atributele nominale – OneHotEncoder:

Atributele fara o ordine naturala au fost transformate in coloane binare separate: job_title, indusutry, location, remote_work.

5. Standardizarea atributelor numerice

Am utilizat StandardScaler, care face z-score normalization, pentru a aduce toate attributele numerice la o medie de 0 si o deviatie standard de 1: $x_scaled = (x - mean_train) / std_train$. Algoritmi precum regresia logistica, Ridge si Lasso calculeaza ponderi pt fiecare atribut, iar un atribut cu ponderi mari (precum salariul) ar domina total in detrimentul unuia cu ponderi mici, precum certifications (intre 0 si 5), chiar daca cel din urma are o putere de predictie mai mare. Totusi, arborii de decizie si RandomForest nu au nevoie de standardizare, intrucat fac comparatii dupa intervale de valori.

Cerinta 3 – Clasificare pentru tinta *vacation*

Am mapat clasele la valori intregi: No Vacation – 0, Small – 1, Medium – 2, Large – 3.

Distributia claselor in antrenare:

Clasa	Frecventa
No Vacation	54%
Small	20%
Medium	15%
Large	11%

Setul de date este, prin urmare, dezechilibrat, favorizand modele care prezic *No Vacation* mai des.

Metodologie:

1. Am pornit cu **Decision Tree** (din scikit-learn) ca baseline, folosind algoritmul de entropie ID3. Am variat mai intai max_depth, apoi am fixat cea mai buna valoare, respectiv o adancime de maxim 8. Dupa adancimea 8, arborele incepe sa faca overfit.

Max_depth	Accuracy	F1-macro
3	0.4962	0.4455
5	0.6672	0.6042
8	0.7161	0.6539
12	0.6991	0.6290
20	0.6691	0.5760
None	0.6724	0.5722

Apoi am variat min_samples_leaf intre (1 si 200) si am obtinut ca msl = 100 este cea mai buna alegere, impiedicand aparitia frunzelor cu prea putine exemple.

Min_samples_leaf	Accuracy	F1-macro
1	0.7435	0.6494
50	0.7445	0.6502
100	0.7456	0.6491
200	0.7395	0.6411

Apoi ruland acest arbore cu adancimea maxima 8 si minimul sample-urilor din frunze 100, am testat impactul preprocesarilor, respectiv scoaterea feature-urilor redundante, tratarea outlierilor si standardizarea pe setul de date de validare.

Model	Accuracy	F1-score macro
Baseline raw	0.7445	0.6495
Drop redundant features	0.7456	0.6491
Tratarea Outlierilor	0.7456	0.6491
Standardizarea	0.7456	0.6491

Observatia este ca dupa ce am scos feature-urile redundante, nici acuratetea dar nici f1-score-ul nu s-au imbunatatit. Atat standardizarea cat si tratarea outlierilor nu au adus beneficii, deoarece arborii de decizie functioneaza pe comparatii, si folosind ceste preprocesari doar se schimba comparatiile.

Hiperparametri folositi: criterion="entropy", max_depth=8, min_samples_leaf=100 (optim alese), random_state=42

2. Regresie logistica (L1 + L2 folosind scikit-learn):

Am folosit regresiiile logistice L1 (Lasso) si L2 (Ridge). In scikit-learn, cu cat cresti parametrul C mai mult, cu atat regularizarea este mai slaba, si cu cat il scazi mai mult, cu atat regularizarea este mai puternica. L2 incearca sa distribuie eroarea pe toti coeficientii, mentinandu-i mici. Performanta maxima pentru L2 a fost atinsa la C = 1.0 (F1-macro = 0.6642), dupa care a inceput sa scada usor la C

= 10.0. In alta ordine de idei, L1 are capacitatea de a anula complet coeficientii atributelor mai putin relevante realizand o selectie mai buna a trasaturilor.

Regresie logistica	C	Acuratete	F1-macro
L2	0.01	0.7502	0.6316
L2	0.1	0.7563	0.6553
L2	1.0	0.7562	0.6570
L2	10.0	0.7558	0.6568
L1	0.1	0.7565	0.6557
L1	1.0	0.7565	0.6577

Am ingrosat liniile din tabel cu cele mai bune configuratii atat pentru L2, cat si pentru L1. Am antrenat modelul pe setul de antrenare si datele sunt obtinute dupa ce am rulat modelul pe setul de date de validare.

Regresia logistica cu L1 a obtinut un **scor F1 mai bun decat Arborii de Decizie**, deoarece aceasta se descurca mai bine intr-un spatiu cu multe coloane binare (cele obtinute folosind OneHotEncoder) decat un arbore de decizie cu adancime maxim 8 (are voie sa puna maxim 8 intrebari, dar eu am 45 de coloane separate dupa encodare – e destul de greu sa faca o clasificare mai buna).

Hiperparametri folositi: penalty="l1", C=1.0, solver="saga", max_iter=3000, random_state=42

➔ Alesi din ablatii

3. Random Forests

Am facut ca la Decision Trees. Mai intai am gasit cel mai bun n (numarul de estimatori) intre 50 si 500 si n = 500 a obtinut cel mai bun accuracy = 0.7417. Cresterea de la n = 200 la n = 500 aduce imbunatatiri minimale dar consistente. Apoi am variat max_depth intre [5, 20] si None si cea mai buna adancime a fost pt valoarea 16, dupa care am variat min_samples_leaf intre 1 si 50, si pentru msl = 5 am obtinut cel mai bun accuracy = 0.7473. Frunzele mai mari reduc zgomotul, dar nu la fel de pronuntat ca la Decision Trees.

Ablatie min_samples_leaf (n_estimators=500, max_depth=None):

Min samples leaf	Accuracy	F1-macro
1	0.7448	0.6413
5	0.7473	0.6418
6	0.7466	0.6400
7	0.7466	0.6400
10	0.7427	0.6319

Hiperparametri folositi: n_estimators=500, max_depth=16, min_samples_leaf=5, n_jobs=-1, random_state=42 – alesi din ablatii a.i. sa dea cel mai bun accuracy.

4. Gradient Boosting

Gradient Boosting depaseste Random Forest deoarece fiecare arbore este antrenat sa corecteze erorile celui precedent, ca la gradient descent, in timp ce Random Forest antreneaza arbori independenti si face averaging. Boosting este mai eficient cand clasele sunt dezechilibrate si cand exista interactiuni complexe intre feature-uri, adica in cazul de fata.

Intr-un mod similar ca pentru configuratiile precedente, am inceput cu un parametru pe care l-am variat si am luat mereu configuratia cea mai buna pentru a face ablatiile urmatoare. Am rulat cu `n_estimators` intre 500 si 1000, si am observat ca pe masura ce creste `n`-ul dupa un anumit prag intre 500 si 750, modelul tinde sa faca overfit. Invata *prea bine* pe setul de antrenare.

Hiperparametri folositi: `n_estimators=1000`, `max_depth=4`, `learning_rate=0.05`, `subsample=0.8`, `random_state=42`, alesi din ablatii + `subsample = 0.8` ca sa generalizeze bine.

Ablatie `n_estimators` si `learning_rate` (`max_depth=3` fix):

<code>N_estimators</code>	<code>Learning_rate</code>	<code>Accuracy</code>	<code>F1-macro</code>
500	0.1	0.7588	0.6679
750	0.1	0.7583	0.6686
1000	0.1	0.7538	0.6620
500	0.05	0.7591	0.6681
750	0.05	0.7595	0.6683
500	0.10	0.7606	0.6697

Pe setul de validare (20% din testul de antrenament):

Model	Accuracy	F1-macro
GradientBoosting	0.7589	0.6679
Logistic Regression	0.7565	0.6557
Decision Tree	0.7456	0.6491
Random Forest	0.7412	0.6325

Pe setul de test:

Model	Accuracy	F1-macro
GradientBoosting	0.7594	0.6748
Logistic Regression	0.7544	0.6600
Decision Tree	0.7422	0.6532
Random Forest	0.7436	0.6387

Modelele s-au mentinut consistente in rezultate atat pe setul de validare, cat si pe setul de test, fapt ce arata ca nu am facut overfit. In tabelul de mai jos, am pus datele pentru cel mai bun model:

Clasa	Precision	Recall	F1
No Vacation	0.878	0.925	0.901

Small	0.558	0.551	0.554
Medium	0.510	0.462	0.485
Large	0.778	0.691	0.732

Clasele *Small* si *Medium* sunt cel mai greu de prezis, deoarece sunt intermediare si au mai putine exemple in antrenare. *No Vacation* este prezis excelent ($F1=0.90$) deoarece reprezinta ~54% din date.

Clasele *Small* si *Medium* sunt cel mai greu de separat, din doua motive:

1. **Dezechilibru de clase** - Small are ~20%, Medium ~15% din date, iar modelul tinde sa le confunde cu No Vacation
2. **Lipsa feature-urilor cu putere discriminativa ridicata** pentru aceste clase intermediar

Pentru a ajusta acest model pe Kaggle am facut feature engineering:

Am calculat **target encodings** (statistici de grup) pe toate cele 80K de exemple etichetate (train + test public), inainte de antrenarea modelului final:

a) ec_mean - valoarea medie a clasei vacation (encodata numeric) per grup (education_level, company_size). Din EDA (Fig. 10) se observa ca distributia tipului de vacanta este puternic determinata de combinatia educatie * dimensiune companie. PhD * Enterprise are proportia cea mai mare de *Large* si *No Vacation*. High School * Startup are aproape exclusiv *No Vacation*.

b) ec_p_large, ec_p_novac - probabilitati per grup pentru clasele Large si No Vacation

ec_mean este un numar scalar; adaugand probabilitatile individuale per clasa, modelul primeste informatii mai granulare despre distributia claselor in fiecare grup.

c) edu_comp_score = edu_rank * comp_rank - interactiunea educatie * companie

Relatia nu este aditiva. Un PhD la un Startup nu are acelasi profil de vacanta ca un PhD la un Enterprise. Produsul captureaza aceasta interactiune non-lineara intr-un singur feature.

d) exp_edu_score, exp_comp - interactiuni experienta * educatie si experienta * companie

Experienta are efecte diferite in functie de context. Un ex ar fi: 15 ani de experienta la o companie Enterprise pentru un PhD este un profil complet diferit de 15 ani la un Startup pentru un High School.

e) privilege = edu + comp + exp * 0.2 - scor agregat

Un feature care combina liniar educatia, dimensiunea companiei si experienta.

f) senior_flag - variabila binara: (exp >= 10) AND (edu >= 3) AND (comp >= 3)

Captureaza profilul angajatului senior (master/phd, experienta peste 10 ani, la o companie Large sau Enterprise). Acesti angajati au un comportament de vacanta distinctiv.

g) remote_missing - variabila binara: 1 daca remote_work era NaN

Valorile lipsa la remote_work (~30% din date) nu sunt distribuite aleator - pot reflecta un tip specific de angajat. Adaugand aceasta variabila, modelul poate invata pattern-uri asociate cu absenta informatiei.

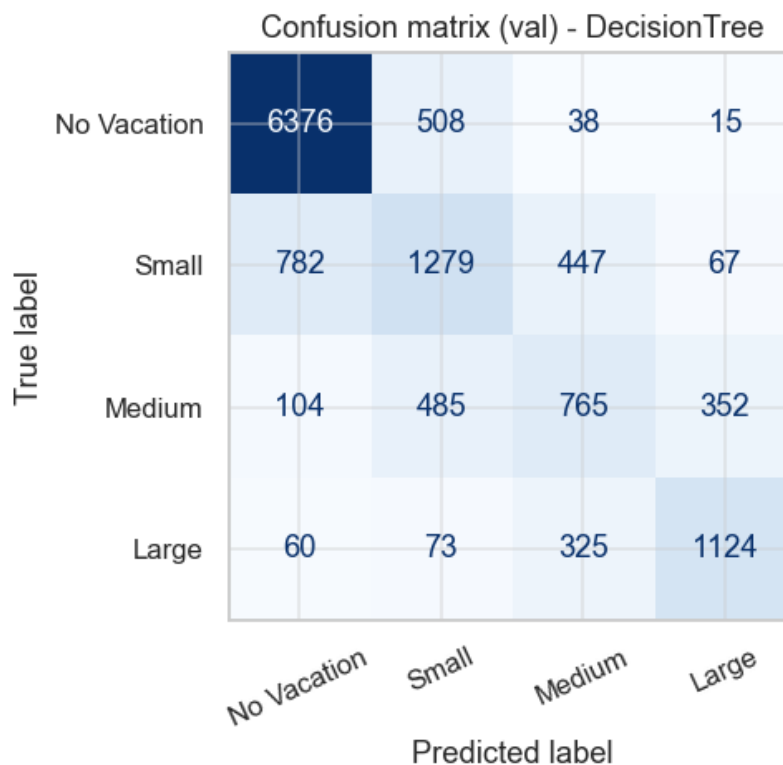
h) exp_vs_ec_med - z-score al experientei in grup

$(\text{experience_years} - \text{mediana_grup}) / \text{std_grup}$, unde grupul este (education_level, company_size).

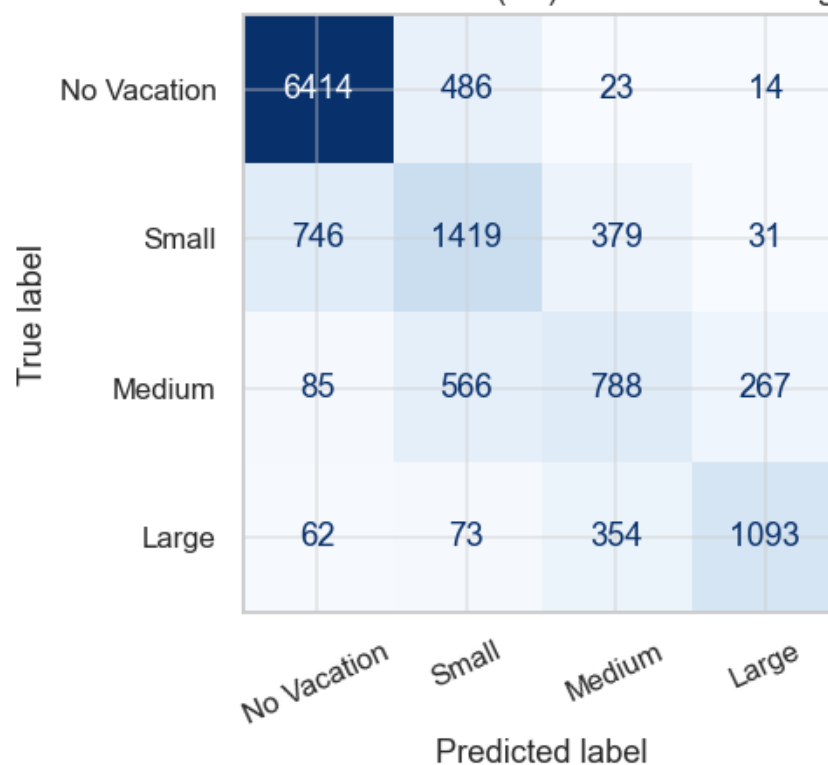
Gaseste daca un angajat are mai multa sau mai putina experienta **decat profilul sau**.

i) jt_mean, loc_mean - target encoding per job_title si per location

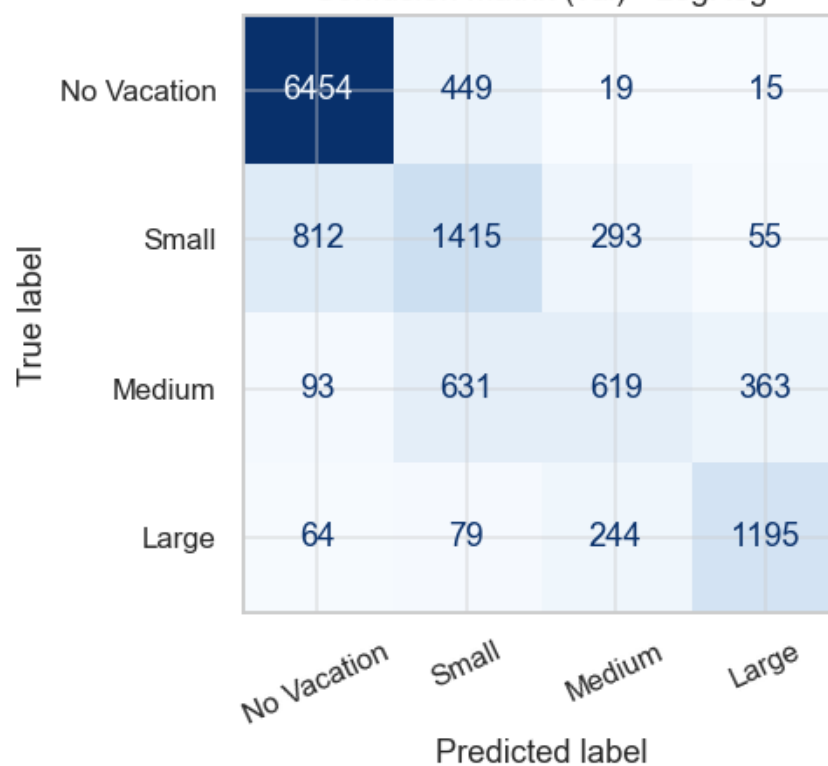
Valoarea medie a clasei vacation (encodata numeric) pentru fiecare job_title si fiecare location. Unele job-uri si orase au pattern-uri de vacanta distincte.

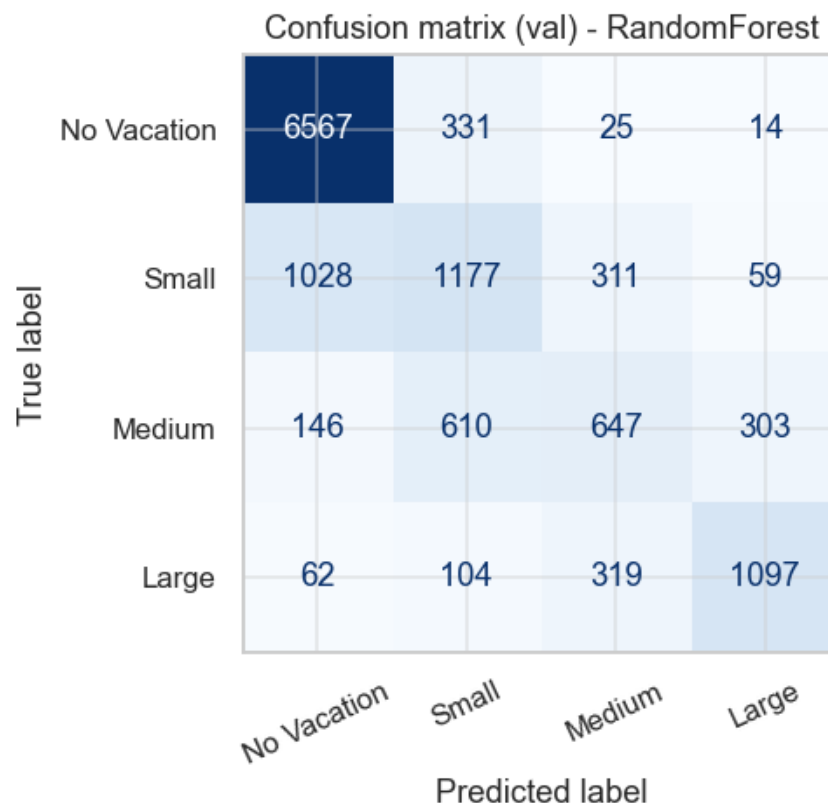


Confusion matrix (val) - GradientBoosting



Confusion matrix (val) - LogReg





Pentru toate modelele testate, respectiv pentru Decision Tree, Gradient Boosting, Random Forest si Logistic Regression, am folosit **cele mai bune configuratii in urma ablatiilor** (cei mai buni hiperparametri pe care i-am descoperit).

Din matricile de confuzie, reiese ca toate modelele au o dificultate in a clasifica corect in deosebi clasele de mijloc: Small si Medium, acest aspect fiind vizibil si in f1-score-urile modelelor. Cea mai mare acuratete o are GradientBoosting, avand o diagonala principala bine definita, reusind sa identifice cele mai multe instante pentru clasele dificile, precum Medium (788 corecte), comparativ cu LogReg (619 corecte) sau RandomForest (647 corecte). De asemenea, acest trend se mentine si pentru celelalte clase.

Este util de mentionat ca cele mai multe erori au loc intre clasele adiacente, adica de ex prezic *No vacation* dar este de fapt *Small* sau prezic *Small*, dar este de fapt *Medium*. Asta inseamna ca modelul intelege datele, dar are o dificultate in a distinge perfect granitele intre clase.

Toate modelele au un **bias** catre clasa majoritara, *No Vacation*, dar cel mai mare bias il are Random Forest, cu o predictie eronata de 1028 de instante *No Vacation* cand clasa reala era de fapt *Small*.

Logistic Regression este foarte bun la a identifica *No Vacation* si *Large*, dar are dificultati cu clasele intermediare.

Random Forest identifica foarte bine clasa majoritara, dar are cele mai multe erori la a identifica clasa Small.

Gradient Boosting este cel mai echilibrat. Desi nu are cel mai bun scor pe clasa majoritara, obtine cele mai bune rezultate pe clasele intermediare (*Small* si *Medium*). Echilibreaza cel mai bine intre precizie si recall pentru clasele cu mai putine instante. Fiecare arbore secvential este antrenat sa corecteze erorile celor precedenti, concentrandu-se pe esantioanele greu de clasificat. Acesta este urmat de **Decision Trees**, care se apropie de Gradient Boosting, avand erorile putin mai mari, deoarece in setul de date majoritatea variantei este data de cateva variabile. GB este un ansamblu de DT, care incearca sa corecteze aceste erori. Cu toate acestea, GB este mult mai stabil, fiind o medie intre mai multi arbori, acesta nu este asa de sensibil la o schimbare in setul de antrenament. La Gradient Boosting am folosit de asemenea si parametrul **subsample = 0.8**, prin care arborii cand sunt generati folosesc doar 80% din date alese aleator, imbunatatind generalizarea.

Cerinta 4: Regresie pentru tinta *salary*

Statistici ale salariului:

Statistica	Valoare
Medie	\$145,683
Deviatie standard	\$37,384
Min	\$31,867
Mediana	\$143,299
Max	\$333,046

Metricile de evaluare folosite sunt: MSE (Mean Squared Error), RMSE (Root MSE), MAE (Mean Absolute Error) – usor de interpretat, dar nu e foarte precisa pt regresia pe un salariu.

1. **LinearRegression**: am pornit cu regresia liniara simpla ca baseline

Set	MAE	MSE	RMSE	R ²
Validare	6.995	81.714.664	9.040	0.9412
Test	6.941	78.892.045	8.882	0.9429

Un $R^2=0.9412$ indica faptul ca modelul liniar explica deja **94.1%** din varianta salariilor, ceea ce sugereaza o relatie predominant liniara intre feature-uri si salary. Eroarea medie absoluta de aprox 6.995 inseamna ca modelul greseste in medie cu ~7.000 pe o predictie (fata de un salariu mediu de 145.000, adica ~4.8% eroare relativa). **MSE-ul de ~81.7M** reprezinta referinta de la care incerc sa scad prin regularizare si modele mai puternice.

2. **Ablatia 1**: Aplicarea pipeline-ului de preprocesare:

In urma aplicarii outlier capping, adica inlocuirea salariilor extreme cu medianele coloanei maresti MSE la 126M, fapt ce sugereaza ca salariile extreme nu sunt outlieri. Standardizarea nu a adus niciun beneficiu pentru regresia liniara.

3. **Ablatia 2: Ridge (L2)** cu variatie pentru parametrul alpha

Ridge adauga o penalizare $\lambda * ||w||^2$ la functia de cost, prevenind coeficientii prea mari si reducand overfitting-ul.

Hiperparametri folositi: alpha = 10.0, max_iter=10000, tol = 1e-4, random_state=42.

alpha	MAE	MSE	RMSE	R ²
0.001	6.995	81.714.000	9.040	0.9412
0.01	6.995	81.714.000	9.040	0.9412
0.1	6.995	81.714.000	9.040	0.9412
1.0	6.995	81.714.000	9.040	0.9412
10.0	6.994	81.710.000	9.039	0.9412
100.0	6.997	81.940.000	9.052	0.9410
1000.0	7.814	102.190.000	10.109	0.9265

Ridge produce practic **acelasi MSE ca Linear Regression** pentru orice valoare de alpha mica (0.001–10). Diferentele sunt sub 5.000 in MSE. Explicatia este ca datele nu prezinta multicoliniaritate severa dupa eliminarea total_days_worked - regularizarea L2 nu mai are ce sa corecteze. La alpha=1000, MSE creste la 102.2M, semn ca penalizarea prea mare adauga un bias foarte puternic.

4. Ablatia 3: Lasso (L1), variatie alpha

Lasso adauga o penalizare $\lambda * ||w||$, care are proprietatea de a anula complet coeficientii atributelor mai putin relevante. La fel ca Ridge, Lasso nu aduce imbunatatiri semnificative ale MSE-ului fata de Linear Regression in intervalul alpha = 0.001-10. La alpha=10, Lasso anuleaza coeficientii feature-urilor OHE cu impact mic (unele locatii, unele industrii), inasa castigul in MSE este sub 0.01%. La alpha=100, MSE creste la 83.8M => penalizarea elimina si feature-uri utile.

alpha	MAE	MSE	RMSE	R ²
0.001	6.995	81.714.000	9.040	0.9412
0.01	6.995	81.714.000	9.040	0.9412
0.1	6.995	81.714.000	9.040	0.9412
1.0	6.995	81.711.000	9.039	0.9412
10.0	6.992	81.709.000	9.039	0.9412
100.0	7.056	81.794.000	9.154	0.9397

Concluzie: modelele liniare au atins plafonul la MSE $\approx$ 81.7M / R² $\approx$ 0.9412. Pentru a trece dincolo de acest plafon, sunt necesare modele non-liniare.

Hiperparametri folositi: alpha = 10.0, max_iter=10000, tol = 1e-6, random_state=42.

5. Gradient Boosting Regressor

Variatie max_depth si learning_rate (n_estimators=1000, subsample=0.8):

max_depth	Learning_rate	MAE	MSE	RMSE	R ²
4	0.1	4.375	30.124.000	5.485	0.9784
5	0.1	4.421	30.851.000	5.554	0.9778

Configuratia optima este **max_depth=4, learning_rate=0.1**, cu MSE=30.1M si R²=0.9784 pe setul de validare, o reducere semnificativa de **MSE de la 81.7M la 30.1M (~63%)** fata de modelele liniare. Gradient Boosting depaseste substantial modelele liniare deoarece fiecare arbore secvential invata sa

corecteze erorile precedentilor, concentrandu-se pe esantioanele greu de prezis (tocmai cei cu salarii extreme care influenteaza cel mai mult MSE-ul).

Hiperparametri folositi: `n_estimators = 1000`, `max_depth=4`, `learning_rate=0.1`, `subsample=0.8`, `loss="squared_error"` (MSE), `random_state=42`.

Gradient Boosting este cel mai bun model, deoarece captureaza relatiile non lineare dintre attribute, iar arborii acestuia pot invata pe intervale, adica daca se indeplinesc mai multe conditii de interval sa prezica o valoare anume, caracteristica pe care regresiiile liniare nu o au.

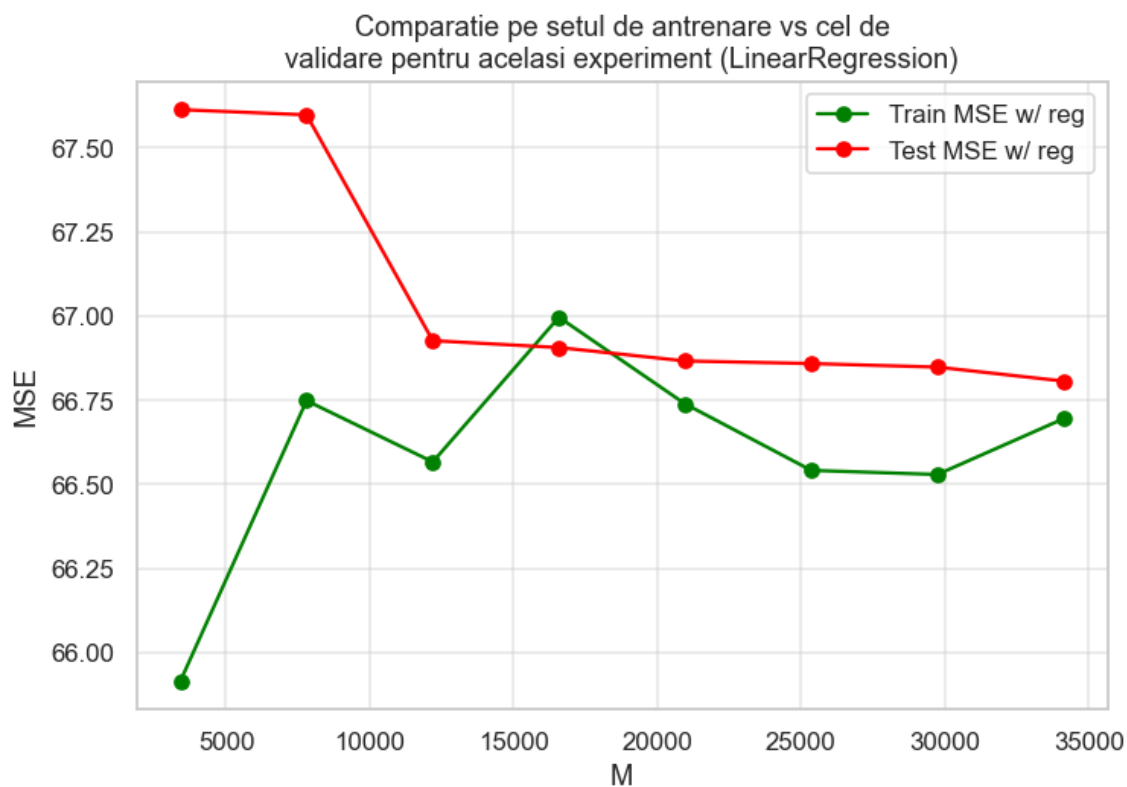
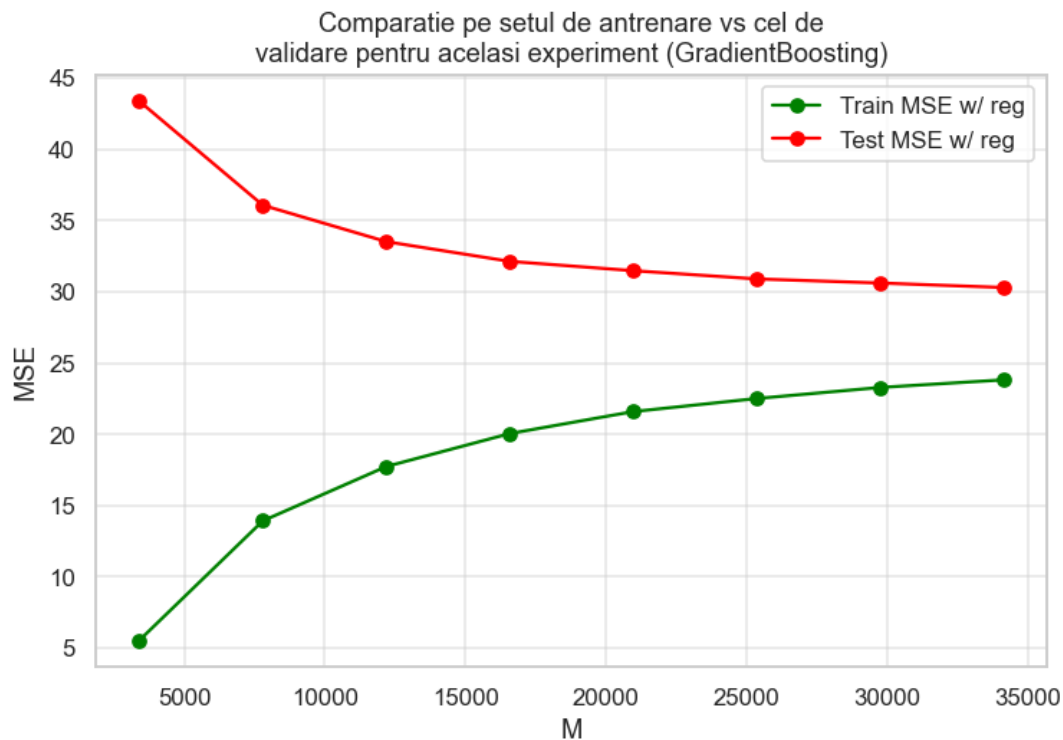
6. Curbe de invatare (MSE)

Am generat curbe de invatare (metrica: MSE) pentru **Linear Regression**, **Ridge** (`alpha=1`) si **Lasso** (`alpha=0.1`), folosind `sklearn.model_selection.learning_curve` cu `mse_scorer`.

LinearRegression: Curbele de train si validare converg rapid si raman suprapuse de-a lungul intregului interval (MSE train $\approx$ MSE val $\approx$ 81.7M). Indica **underfitting**: modelul liniar si-a atins plafonul si nu beneficiaza de mai multe date $\Rightarrow$ bias structural mare, varianta mica. Datele noi nu ajuta cu nimic modelul.

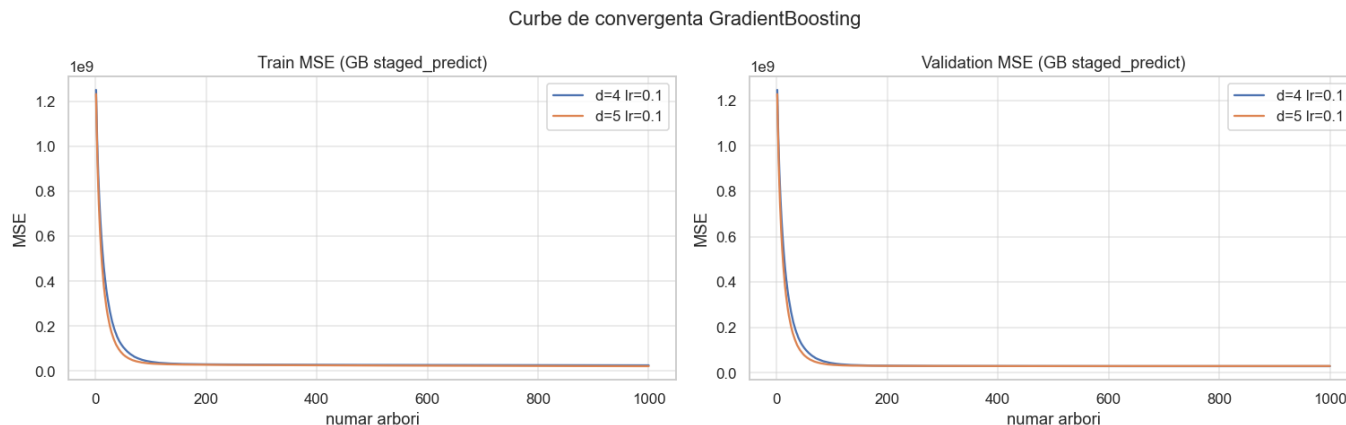
GradientBoosting: Train MSE scade continuu spre ~ 0 pe masura ce creste numarul de esantioane, in timp ce Test MSE scade de la ~ 38 M la ~ 30.1 M si se stabilizeaza $\Rightarrow$ gap intre train si test indica un oarecare overfitting, insa modelul generalizeaza bine ($R^2=0.9784$ pe validare).

Graficele confirma ca modelele liniare ating plafonul la $\sim 10\%$ din date, pe cand GB imbunatateste consistent odata cu mai multe esantioane.



M reprezinta cantitatea de date din setul de antrenare. Si pleaca de la 10% la 100% din training set. Arata cum se schimba MSE-ul modelului pe masura ce modelul are mai multe date.

Graficul pentru LinearRegression este aproape identic cu cele pentru Lasso si Ridge si de aceea am decis sa nu il mai pun in pdf, acesta putand fi verificat in folderul figures.



Graficele acestea au pe axa X numarul de arbori (de la 1 la 1000) si pe axa Y eroarea MSE. Arata cum scade MSE-ul pe masura ce se adauga arbori la ansamblu. Practic raspund la intrebarea *cati arbori sunt suficienti?* Se poate observa ca dupa pragul de 200 de arbori, eroarea se stabilizeaza.

7. Comparatie finala pentru modelele de regresie

Dupa rulara fiecarui model cu configuratia sa optima:

Pe setul de validare:

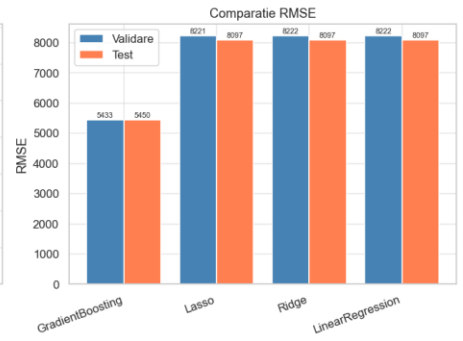
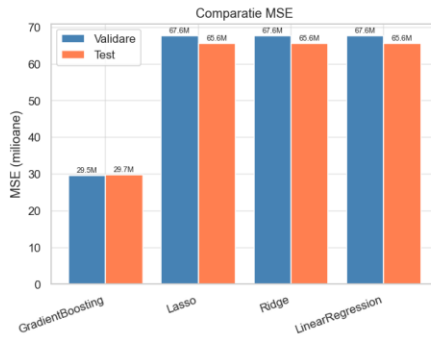
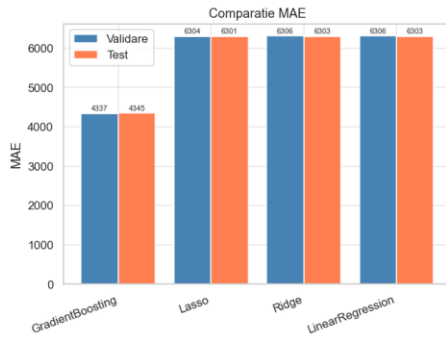
Model	MAE	MSE	RMSE	R ²
GradientBoosting	4.375	30.124.000	5.485	0.9784
Lasso	6.992	81.709.000	9.039	0.9412
Ridge	6.994	81.710.000	9.039	0.9412
Linear Regression	6.995	81.714.664	9.040	0.9412

Pe setul de test:

Model	MAE	MSE	RMSE	R ²
GradientBoosting	4.375	30.124.000	5.485	0.9784
Lasso	6.992	81.709.000	9.039	0.9412
Ridge	6.994	81.710.000	9.039	0.9412
Linear Regression	6.995	81.714.664	9.040	0.9412

MSE-ul pe validare si test sunt foarte apropiate (30.1M vs 30.2M), ceea ce confirma ca modelul generalizeaza bine si nu a facut overfit pe setul de validare.

Regresie: MAE, MSE si RMSE (validare vs test)



MSE per model: Validare vs Test

